Hugging Face Hub end-to-end Tutorial

- *Sections 2 to 4 are documented in the Jupyter Notebook `1_iris_classification.ipynb`*
- *Section 5 in `2_upload_dataset.ipynb`*
- *Section 6 in `3_upload_model.ipynb`*
- *Section 7 in script `4_gradio.py`*

The notebook `1_iris_classification.ipynb` trains a Decision Tree classifier. The tutorial contains the modified code for the Logistic Regression model

- **YOUR TASKS for accomplishing this tutorial** are:
  1. Adapt the code in the notebook for this other classifier (**section 4**)
  2. Add the new Logistic Regression model to the same Hugging Face model repository, giving the option to the user of using one or the other (**section 6**)

NOTE: *The code snippets in this tutorial are minimal and orientative. Refer to Jupyter Notebook `1_iris_classification.ipynb` for the complete code.*

1. Project Setup

1. **Create/Activate a Python Environment**
   - Install required packages with pinned versions to ensure reproducibility:

```
pip install scikit-learn==1.8.0 huggingface_hub==0.36.0 datasets==4.4.2
gradio==6.9.0 joblib==1.5.3
```

   Or install from `requirements.txt`:

```
pip install -r requirements.txt
```

   > **Note:** Always pin dependency versions. Libraries like Gradio introduce breaking changes between major versions (e.g. flagging behaviour changed in v4+).

2. **Authenticate with Hugging Face**
   - Create an access token on Hugging Face
     - *Settings* > *Access Tokens*: Create new token, Write type
   - Use `huggingface-cli login` and paste your token
   - Alternatively, login from a Python script:

```
    import huggingface_hub
    huggingface_hub.login(token=...)`
```

2. Loading the Iris Dataset

You have Iris dataset available in path `./iris-HF/Iris.csv`, so you do not need to obtain it externally. For your reference, here are two ways to get the dataset using a Hugging Face repository ot the `datasets` library (equivalent to importing from Sklearn).

1. Clone the dataset from Hugging Face hub:

```
git lfs install
git clone https://huggingface.co/datasets/scikit-learn/iris
```

2. If you are inside Python you can alternatively use `datasets` HF library:

```
from datasets import load_dataset
```

Or load data from Scikit-Learn's built-in dataset:

```
from sklearn.datasets import load_iris
data = load_iris()
```

Afterwards inspect the Data:

- Print feature names, shapes, target distribution, etc.

3. Preparing and Splitting the Data

1. **Train/Test Split**
   - Use `train_test_split` from `sklearn.model_selection`.
   - Example:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    data['data'], data['target'], test_size=0.2, random_state=42
)
```

2. **Preprocessing**
   - Scale features using `StandardScaler`. Logistic Regression is sensitive to feature scale, so wrapping the scaler and classifier in a `Pipeline` is the recommended approach — it ensures scaling is applied consistently and prevents data leakage:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", your_classifier)
])
```

---

4. Training a Logistic Regression Model

1. **Train the Model** Use a `Pipeline` to include feature scaling (required for Logistic Regression):

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

pipeline_logreg = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=200))
])

pipeline_logreg.fit(X_train, y_train)
```

> `max_iter=200`: the default (100) is sometimes insufficient for convergence on this dataset.

2. **Evaluate the Model**
   - Generate predictions, compute accuracy:

```
from sklearn.metrics import accuracy_score, classification_report

y_pred = pipeline_logreg.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

3. **Save the Model**

```
import joblib
joblib.dump(pipeline_logreg, "models/iris_logreg.joblib")
```

---

5. Uploading the Dataset to Hugging Face

Refer to script `2_upload_dataset.ipynb`

1. **Create a Dataset Repository**

   - Using `huggingface_hub` to initialize a repo:

```
from huggingface_hub import HfApi

api = HfApi()
repo_id = "username/iris-dataset"
api.create_repo(repo_id=repo_id, repo_type="dataset")
```

2. **Push the Data Files**

   - Store the split data in CSV or Parquet.
   - Push the CSV file of the dataset:

```
 api.upload_file(
 path_or_fileobj="iris-HF/iris.csv",        # Local path to your CSV
 path_in_repo="iris.csv",           # File name/path in the repository
 repo_id= repo_id,     # The dataset repository on HF
 repo_type="dataset",               # repo_type="dataset" for a dataset repo
 # token="hf_your_token_here"         # Your personal HF token (not needed,
because you're already logged in)
)
```

   Or using the command line:

```
huggingface-cli upload brjapon/iris . --repo-type=dataset
```

3. **Dataset Card**

   - Clone the repository locally:

```
   git clone https://huggingface.co/datasets/USERNAME/iris
```

   - Create a `README.md` describing the dataset.
   - Push it from the command line using git

```
git add README.md
git commit -m "Added README file"
git push
```

---

6. Saving and Uploading the Model to Hugging Face

Refer to script `3_upload_model.ipynb`

Both models (Decision Tree and Logistic Regression) are stored in the **same model repository** so users can choose between them from a single location.

1. **Serialize Your Models**

```
import joblib
joblib.dump(pipeline_dt,     "models/iris_dt.joblib")
joblib.dump(pipeline_logreg, "models/iris_logreg.joblib")
```

2. **Create a Model Repository** (once)

```
from huggingface_hub import HfApi

api = HfApi()
repo_id = "username/iris-dt"
api.create_repo(repo_id=repo_id, repo_type="model")
```

3. **Push Both Models to the Same Repo**

```
api.upload_file(
    path_or_fileobj="models/iris_dt.joblib",
    path_in_repo="iris_dt.joblib",
    repo_id=repo_id,
    repo_type="model"
)

api.upload_file(
    path_or_fileobj="models/iris_logreg.joblib",
    path_in_repo="iris_logreg.joblib",
    repo_id=repo_id,
    repo_type="model"
)
```

4. **Model Card**
   - Clone the repository locally:

```
   git clone https://huggingface.co/models/USERNAME/iris-dt
```

   - Create a `README.md` describing the models.
   - Push it from the command line using git

```
git add README.md
git commit -m "Added README file"
git push
```

---

7. Creating a Hugging Face Space for Deployment

Refer to script `4_gradio.py`

1. **Choose a Framework (Gradio/Streamlit)** We use Gradio, which is focused on Machine Learning apps. The final version of `4_gradio.py`:
    - Downloads both models from HF Hub at startup (cached locally after first run)
    - Lets the user select which model to use via a dropdown
    - Enables flagging so users can mark incorrect predictions for later review

```
import gradio as gr
import joblib
import numpy as np
from huggingface_hub import hf_hub_download

# Download both models from HF Hub at startup (cached after first run)
dt_path = hf_hub_download(repo_id="brjapon/iris-dt",
filename="iris_dt.joblib", repo_type="model")
lr_path = hf_hub_download(repo_id="brjapon/iris-dt",
filename="iris_logreg.joblib", repo_type="model")

models = {
    "Decision Tree": joblib.load(dt_path),
    "Logistic Regression": joblib.load(lr_path),
}

LABELS = {0: "Iris-setosa", 1: "Iris-versicolor", 2: "Iris-virginica"}

def predict_iris(model_choice, sepal_length, sepal_width, petal_length,
petal_width):
    pipeline = models[model_choice]
    input = np.array([[sepal_length, sepal_width, petal_length,
petal_width]])
    prediction = pipeline.predict(input)
    return LABELS.get(int(prediction[0]), "Invalid prediction")

interface = gr.Interface(
    fn=predict_iris,
    inputs=[
        gr.Dropdown(choices=["Decision Tree", "Logistic Regression"],
label="Model", value="Decision Tree"),
        gr.Number(label="Sepal Length (cm)"),
        gr.Number(label="Sepal Width (cm)"),
        gr.Number(label="Petal Length (cm)"),
```

```
        gr.Number(label="Petal Width (cm)"),
    ],
    outputs="text",
    live=True,
    title="Iris Species Identifier",
    description="Choose a model and enter the four measurements to predict
the Iris species.",
    flagging_mode="manual",
    flagging_dir="flagged"
)

if __name__ == "__main__":
    interface.launch()
```

> **Gradio 6.x note:** Flagging is disabled by default. You must pass `flagging_mode="manual"` explicitly. Flagged inputs are saved to `flagged/dataset1.csv` (the filename changed from `log.csv` in older versions).

2. **Deploying to Spaces**
   - Create a new Space on Hugging Face (click "New Space," choose Gradio)
   - Clone the repository:

```
# When prompted for a password, use an access token with write permissions
git clone https://huggingface.co/spaces/brjapon/iris-space
cd iris-space
```

   - Copy the Gradio script (Spaces expects it named `app.py`) and the pinned `requirements.txt`:

```
cp ../HuggingFaceHub-Quick-Start/4_gradio.py app.py
cp ../HuggingFaceHub-Quick-Start/requirements.txt requirements.txt
```

   - Push your code:

```
git add app.py requirements.txt
git commit -m "Add model selector and fix flagging"
git push
```

   - Wait for the build to finish, then test your live app

---

8. Testing and Sharing Your Deployed App

1. **Test the UI**
   - Manually input values.

- Verify correctness of predictions.
- Test both models using the dropdown selector and confirm they agree on most inputs.
- Click the **Flag** button on an interesting or incorrect prediction — verify it appears in `flagged/dataset1.csv`.

2. **Share the Space URL**
   - A link for sharing the app can be obtained from the `Share via Link` button that you will see in the live app:
     - https://brjapon-iris-space.hf.space/?__theme=system&deep_link=TFBJ-sNX6lk
   - Collaborators and other users can directly interact with or fork your Space.